

OSSMTWC — Developer Onboarding

What this project is

OSSMTWC is a NetSuite customization built on top of **OSSM's proprietary in-house framework** (SuiteBundles/**Bundle 548734** — installed natively in the NetSuite account, not fully complete, and reused across other OSSM client projects). This repo is the **TWC-specific application** built on that framework: it manages site/tower infrastructure, equipment, contractor accreditation, and a set of paper-form-replacement workflows (Space Request, Site Access, Site Data Sheets).

Business requirements, schemas, and sample data live in `.docs/specs/` and `.docs/data/` — read these to understand *why* a data object looks the way it does; they're the source of truth for the domain model.

Not a standard SDF project

`manifest.xml` , `deploy.xml` , and `.project` are **Eclipse NetSuite IDE plugin** project scaffolding, not SuiteCloud CLI (SDF) config — there's no `suitecloud.config` and the SuiteCloud CLI isn't used here.

Only **one** NetSuite object is tracked in source control:

`Objects/customscript_oTWC_controls_sl.xml` (a Suitelet script/deployment record, not a custom record definition). Everything else — custom record types, NetSuite workflows, roles, script deployments, saved searches — lives natively in the NetSuite account, managed by hand through the NetSuite UI. This repo is effectively a **File Cabinet code sync**: it tracks

FileCabinet/SuiteScripts/OSSMTWC/ , which NetSuite executes directly as SuiteScript.

Deployment is done by pushing individual files into the NetSuite File Cabinet, via one of:

- **VS Code "nsLink" extension** — the team's normal workflow. Configured per-workspace in `.vscode/settings.json` (gitignored — never commit it) with OAuth 1.0 TBA credentials (`nsLink.nsAccountId` , `nsLink.nsConsumerKey/Secret` , `nsLink.nsTokenKey/Secret`). Ask a teammate for your own per-account, per-role token — don't reuse someone else's. You'll typically keep separate `settings.SB.json` / `settings.live.json` variants for sandbox vs. production targets.
- **Eclipse NetSuite IDE plugin**, using `deploy.xml` (deploys `AccountConfiguration/*` , `FileCabinet/*` , `Objects/*` wholesale).

There's no CI/build step and **no local install or test runner** — code is validated by pushing to a sandbox account and exercising it there.

Repository layout

FileCabinet/SuiteScripts/OSSMTWC/

o/ Extensions to the proprietary framework (Bundle 548734) – see below

data/ Business-object + generated-persistent record layers, plus data-layer utility/UI-helper modules

data_nx/ Static reference data not modeled as NetSuite records – IGNORE, not relevant

modules/ Program-workflow engines (SRF, SDS, accreditation)
– NOT NetSuite workflows

svc/ Scheduled-script batch jobs, thin wrappers
delegating to modules/ engines

ui/ Customer-facing feature pages:
Suitelet+ClientScript pairs at root, views/js/modules/css
subfolders

<root> Suitelet/ClientScript or UserEvent/ClientScript
pairs with simple, page/record-local NetSuite logic

Objects/ IGNORE – one tracked script/deployment
record; all custom records live only in NetSuite

XML/ IGNORE – self-explanatory (print template
XML)

FileCabinet/Templates/ IGNORE – not relevant (empty template
folders)

.docs/ Business specs, schemas, sample data,
HOWTO.txt, TODO.txt

The "O" framework (o/)

o/ holds this project's **extensions on top of Bundle 548734**, not the framework itself — the bulk of the framework (core.js , core.sql.js ,

`data/rec.utils.js` , `ui/nsSuitelet.js` , etc.) lives only in the NetSuite File Cabinet under `SuiteBundles/Bundle 548734/` , browse it there in your sandbox if you need to see what a TWC file calls into. Nearly every file here loads bundle modules via `@NAmdConfig / define()` paths like `/SuiteBundles/Bundle 548734/0/... .`

- `0/data/oTWC_baseRecord.js` — `RecordBase` , the ORM base class (see next section).
- `0/controls/oTWC_ui_*.js` — one file per UI widget type (panel, table, `propTable`, `tabControl`, chart, calendar, `dropDown`, toggle, button, `fieldPanel`, input, etc.), funneled through `0/controls/oTWC_ui_ctrlBase.js` . Controls render as a custom `<ossm data-type="...">` element with a base64-encoded JSON config blob for client-side re-hydration.
- `0/oTWC_dialogEx.js` — dialog helpers (confirm/error/message/wizard/site-finder autocomplete).
- `0/oTWC_themes.js` (+ `oTWC_themes_ue.js`), `0/oTWC_userPref.js` , `0/oTWC_nsFileUtils.js` — theming, per-user preferences, File Cabinet helpers.
- `0/css/html.styles.css` .

The ORM pattern (`data/`)

Every custom record follows a three-layer chain:

Suitelet / svc code

→ business object data/oTWC_<name>.js

(hand-edited)

→ persistent record data/persistent/oTWC_<name>Persistent.js

(generated)

→ RecordBase O/data/oTWC_baseRecord.js

(framework)

→ Bundle 548734's rec.utils.js / core.sql.js → N/record

/ SuiteQL

- **Persistent layer** (data/persistent/oTWC_*Persistent.js , 30 files) is **generated by a VS Code extension tool** ("NSUtils - Build Object (v2)") — never hand-edit it, it can be overwritten at any moment. It defines the field-id map (_recordFields), field metadata (_recordFieldInfo), and camelCase get/set accessors wrapping the base class.
- **Business object** (data/oTWC_*.js) is hand-written — the business logic layer — typically class OSSMTWC_X extends xPersistent.PersistentRecord { ... } re-exporting Type / Fields , plus business-specific enums/helpers. Example pairs: oTWC_site.js ↔ persistent/oTWC_sitePersistent.js , oTWC_company.js ↔ persistent/oTWC_companyPersistent.js , oTWC_srf.js ↔ persistent/oTWC_srfPersistent.js .
- RecordBase.select(options) is a hand-rolled SQL query builder (fields / where / orderBy / minimal / noAlias / returnFirst) — this is how most list/search queries are written here, not N/search .
- data/ also holds **utility and server-side UI-helper modules that don't follow the object/persistent pattern** — don't expect a *Persistent.js counterpart for these: oTWC_utils.js , oTWC_icons.js , oTWC_config.js , oTWC_configUIFields.js , oTWC_permissions.js , oTWC_rolePermission.js , and the *UI.js files (oTWC_companyUI.js , oTWC_equipmentUI.js , oTWC_siteUI.js , oTWC_srfUI.js , etc.) which

hold server-side rendering/table logic for a record's UI, separate from its persistence.

- Two business objects currently have **no** generated persistent counterpart — treat as exceptions, confirm with the team before assuming standard behavior: `data/oTWC_siteLevel.js` , `data/oTWC_siteRow.js` .

Feature-slice anatomy (`ui/`)

Each customer-facing page follows the same shape — `ui/oTWC_siteInfo_*` is a good reference:

1. `ui/oTWC_<feature>_cs.js` (ClientScript, at `ui/` root) — thin bootstrapper, calls into the page controller's `init()` .
2. `ui/js/oTWC_<feature>.js` — page controller class extending `TWCPageBase` (`ui/js/oTWC_pageBase.js`): page container, base64-decoded page-data blob, AJAX helpers back to the Suitelet, shared error reporting.
3. `ui/oTWC_<feature>_sl.js` (Suitelet, at `ui/` root) — `get()` builds page data (via `twcBaseView.initPageData()` for permission/user/theme bootstrap) and renders the view; `post()` dispatches on an `action` param to a handler.
4. `ui/views/oTWC_<feature>.html` — a token-substitution template (`{TOKEN}` placeholders, no real templating engine).
5. `ui/modules/oTWC_<feature>Utils.js` — server-side query/render/save logic for the feature.

Plus shared assets: `ui/css/oTWC.css` , `ui/js/oTWC_googleMap.js` , `ui/views/oTWC_baseView.js` / `_ue.js` and `oTWC_pageBase.html` . Cross-cutting concerns (file upload/preview, site search autocomplete) go through the shared `oTWC_microSvc_sl.js` Suitelet (at the `OSSMTWC/` root) rather than being duplicated per feature — treat it as this app's internal AJAX API.

Current feature pages: Company Profile, Dashboards (has a Portlet, `_p1.js`), Inventory, Nexus Record, Site Access, Site Info, Site Locator, Space Request, Trouble Ticket — plus a standalone print-only Suitelet `oTWC_print_srf_sds_sl.js` (no ClientScript pair).

Workflow/engine modules (`modules/`)

These drive **program workflow** (business process state), not NetSuite's own workflow feature:

- `oTWC_srfWorkflowEngine.js` (server-side state machine) + `oTWC_srfWorkflowEngineUI.js` (client-side rendering) — drives the SRF (Space Request Form) approval workflow. Stages/forms are **data-driven**: each stage item stores a `form_data` JSON blob rendered dynamically, and next-stage branching (approve/reject-with-feedback) is computed from `next_stage_pick` config rather than hardcoded per stage.
- `oTWC_certStatusEngine.js` — accreditation/certification expiry roll-up logic (per-person cert statuses → overall status; per-company insurance certs → overall status). Invoked by `svc/oTWC_certStatus_svc.js` .
- `oTWC_sdsEngine.js` / `oTWC_sdsEngineUI.js` / `oTWC_sdsRender.js` — SDS (Site Data Sheet) document generation and rendering (drawing/license file lookup, fibre-provider and agreement-template dialogs, PDF/HTML rendering via `N/render`), tied into the SRF workflow UI.

`svc/` — scheduled batch jobs

`oTWC_certStatus_svc.js` , `oTWC_infra_sts_update_svc.js` , `oTWC_utils_svc.js` — all `@NScriptType ScheduledScript` , thin wrappers delegating to `modules/` engines. They follow a consistent self-throttling pattern: check `runtime.getCurrentScript().getRemainingUsage()` , and if governance is running low, requeue via `N/task` with a resume cursor (last processed id, or a JSON script-parameter payload). Follow this pattern for any new batch job.

Conventions

- **SuiteScript 2.1 everywhere** (`@NApiVersion 2.1`) — no 2.0 files.
- File-suffix naming tells you the script type: `_ue` = User Event, `_cs` = Client Script, `_sl` = Suitelet, `_svc` = Scheduled Script, `_pl` = Portlet. Files without a suffix under `data/` are business objects/persistent records/utilities; under `ui/js/` and `ui/modules/` they're plain library modules.
- In-code review/tracking tags (grep for these before/after a review):
 - `@@TODO` — work still to do
 - `@@NOTE` — explains something non-obvious
 - `@@IMPORTANT` — warns about code that's easy to break by changing
 - `@@REVIEW` — works, but should be optimized or moved into the SDK/bundle
 - `@@CODE-REVIEW` — flagged for review with Giuseppe / Carlos
 - `@@HARDCODED` — must be specified by every hard-coded value
- Git: single `master` branch, direct commits (no PR workflow). Commit messages are terse and inconsistent — don't rely on git history as documentation; the `@@TODO` / `@@REVIEW` tags and `.docs/TODO.txt` are the team's actual tracking mechanism.

Common workflows (from `.docs/HOWTO.txt`)

Add or edit a data object:

1. Export the record's XML from NetSuite.
2. Run the VS Code extension tool "NSUtils - Build Object (v2)" → save output to `data/persistent/oTWC_<name>Persistent.js` .

3. If you only changed an existing object, stop here. For a *new* object, manually create the business object at `data/oTWC_<name>.js` extending the generated persistent class.

Add a feature (new Suitelet page):

1. Create `ui/oTWC_<feature>_sl.js` , `ui/oTWC_<feature>_cs.js` , `ui/js/oTWC_<feature>.js` , `ui/modules/oTWC_<feature>Utils.js` .
2. Deploy the Suitelet in NetSuite and note its script id. Set the external role "TWC Customer Center" as an available role on the deployment if it should be customer-visible.
3. Add a row to the custom record table **"OSSMTWC - Role Feature List"** and update the relevant role(s) — features are gated per-role through this table, not just through Suitelet deployment permissions.

Known gotchas

- `data/persistent/oTWC_rowPersistent.js` (~lines 74-96) has a bad generated find/replace: several field-id references (`ROW_TYPE` , `ROW_REGISTERED` , `ROW_FOLIO` , `ROW_CONDITIONS`) were mangled into `R.O.W._TYPE` etc. inside getter/setter bodies — throws at runtime if those accessors are called. Needs a fix and a regeneration sanity-check next time the "Build Object" tool is used.
- `data/oTWC_siteLevel.js` and `data/oTWC_siteRow.js` have no generated persistent counterpart — don't assume they behave like standard business objects; check with the team first.
- Field re-labelling in NetSuite causes issues when a data object is re-built by the tool.
- `DropDown` 's `noAutoSelect` option doesn't really work as intended.
- `RecordBase.select` 's `noAlias` and `useNames` options are redundant (same thing) — `useNames` should be removed.

- **Security note** (tracked in `.docs/TODO.txt`): the "Radix Role" custom field on customer/vendor records is not restricted from being set to an internal Admin-level role — open risk, not yet mitigated (can't use Sourcing on the field since "Center Type" isn't available as a List/Record option).
- Large file uploads can make the SRF UI slow to save (also tracked in `.docs/TODO.txt`).
- Nexus records: role-based view/edit access (0/1/2) needs to be enforced via NetSuite Forms plus a UserEvent script blocking records a role shouldn't see — not yet implemented, tracked in `.docs/TODO.txt`.

Where to look next

- `.docs/HOWTO.txt` — the two workflows above, verbatim, plus the comment-tag legend.
- `.docs/TODO.txt` — current open issues and framework-level TODOs.
- `.docs/specs/` and `.docs/data/` — business requirements, object schemas, and sample/test data per module (Sites, Accreditation, Pricing, SDS, SRF/SAF).